

Properties of Depth - First Search

- Complete? No, if b is finite but m is infinite. The search will have to be modified to avoid repeated states along the path. This modification will create a set of finite spaces which will make the search complete.
- Time Complexity? $O(b^m)$. It is terrible if m is much larger than d
- Space Complexity? $O(bm)$. It is linear and only needs to store $b \times m$ nodes
- Optimal? No. It might not find the shortest or optimal path. It might not recover from a bad early choice in a large or infinite search space.

Uniform Cost Search

As mentioned before, a data structure of nodes should contain information of path costs.

In some problems, the cost of expanding from one node to its successors might be different. The cost of each path should be maintained, which will give the sum of costs along the path.

Uniform cost search expands the node with the smallest path cost. A function which assigns a cost to a path is called a path cost function. This is often denoted as $g()$.

BFS can be viewed as a special case of uniform cost search.

If the costs of going from one node to the successor nodes on the next level are always the same, then uniform cost search is equivalent to BFS.

In other words, BFS is simply Uniform Cost Search where the cost function

$$g(n) = \text{depth}(n)$$

It is complete. It is also optimal because it selects least-cost unexpanded node so a solution with the shortest path can be found.

Question: How about the time/space complexity of uniform cost search?

Depth-Limited Search

This is the same as a Depth-First search with a depth limit l . This means that the nodes at depth l have no successors.

Recursive implementation:

```
function DEPTH-LIMITED-SEARCH (problem, limit) returns soln/fail/cutoff
    RECURSIVE-DLS (MAKE-NODE(INITIAL-STATE [problem]), problem, limit)
function RECURSIVE-DLS (node, problem, limit) returns soln/fail/cutoff
    cutoff-occurred?  $\leftarrow$  false
    if GOAL-TEST (problem, STATE [node]) then return node
    else if DEPTH [node] = limit then return cutoff
    else for each successor in EXPAND (node, problem) do
        result  $\leftarrow$  RECURSIVE-DLS (successor, problem, limit)
        if result = cutoff then cutoff-occurred?  $\leftarrow$  true
        else if result  $\neq$  failure then return result
    if cutoff-occurred? then return cutoff else return failure
```

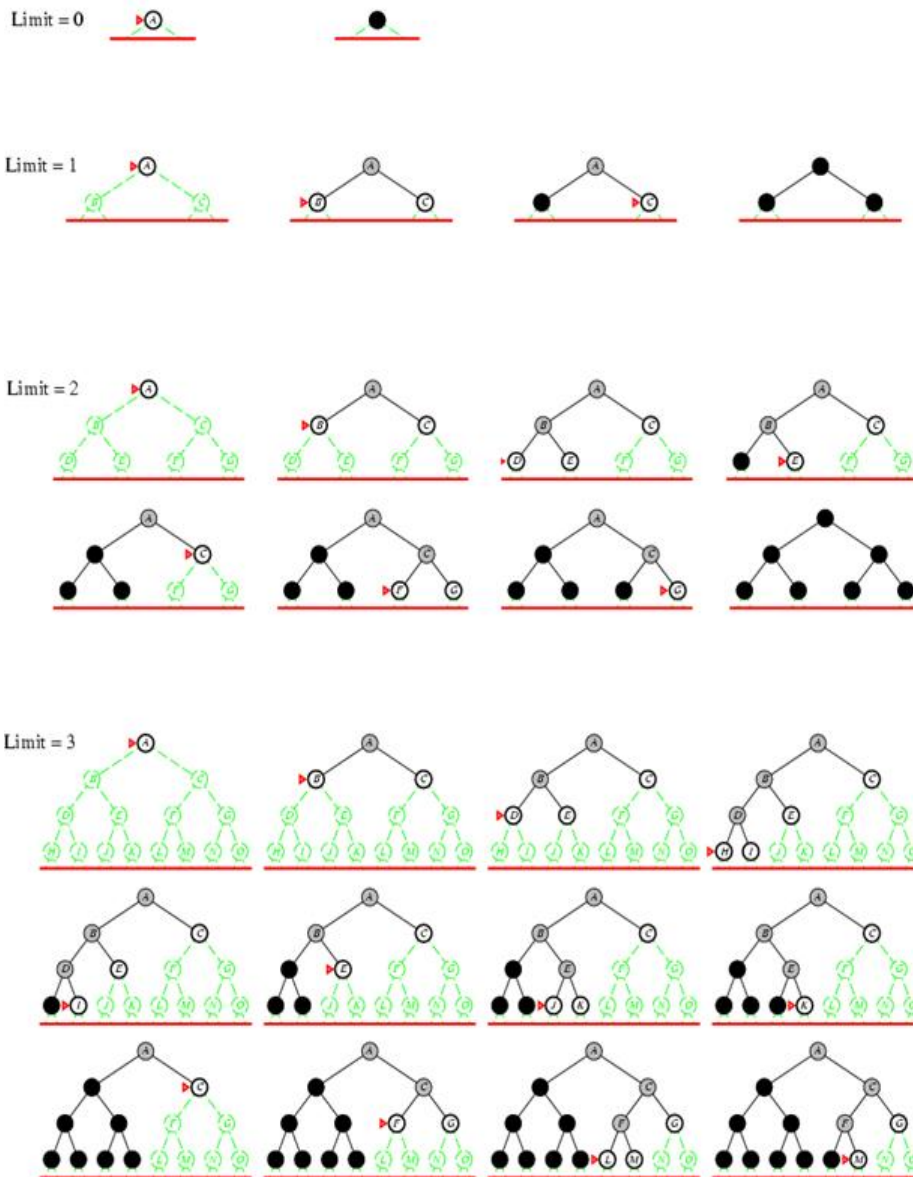
Iterative Deepening Search

As discussed previously, BFS is complete and optimal but has a big problem in space complexity. DFS is not complete and not optimal. It however has linear space complexity and could be faster than BFS.

Iterative deepening combines the benefits of BFS and DFS. It performs Depth First Search with a depth limit. It tries all possible depth limits in an iterative process, and deepens the depth of the search after every iteration.

```
function ITERATIVE-DEEPENING-SEARCH (problem) returns a solution
    inputs: problem, a problem
    for depth  $\leftarrow$  0 to  $\infty$  do
        result  $\leftarrow$  DEPTH-LIMITED-SEARCH (problem, depth)
        if result  $\neq$  cutoff then return result
    end
```

Illustration of Iterative Deepening Search



Properties of Iterative Deepening Search

It is complete and optimal. It also has linear space complexity.

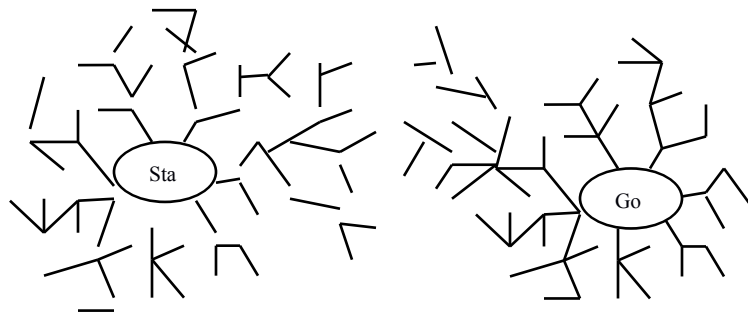
It may seem costly because many nodes are expanded multiple number of times.

This is however not a serious matter because most of the nodes on the tree are at bottom level. They therefore do not add much extra cost if the nodes on the upper levels are expanded several times.

Bi-Directional Search

Bi-directional search simultaneously searches both forward from the initial state, and then backward from the goal state. It stops when the two searches meet in the middle.

The complexity is $O(b^{d/2})$ because both the forward and backward searches each have to go only halfway.



There are however several issues of bi-directional search.

- Bi-directional search must be able to generate predecessors to enable the backward search. In some problems however, this can be very difficult and often times, impossible.
- It is difficult to deal with bi-directional search when there are many possible goal states. Ideally, a problem should have one or a few goal states, but we don't always get ideal situations.
- There must be an efficient way to check whether each new node has already been visited from the other half of the search.
- Also, we need to decide what kind of search is going to take place in each half of the search.

- In order to check whether the two searches meet, the nodes of at least one of them must be retained in memory. This means that the space complexity might be impractical.

Repeated states

In the cats and mice problem, examples of repeated states (states already visited) have been shown. The search algorithms we have looked at however do not deal with repeated states.

Avoiding repeated states is an important aspect of the search process. Various choices can be made depending on the search space.

- Not checking for repeated states at all
- Not returning to the state we have just come from
- Not creating paths with cycles in them
- Not generating any state that has already been generated

Question: What kind of data structure will be best suited for checking repeated states?

Constraint Satisfaction

In some problems, the states are defined by the values of a set of variables and these values must obey certain *constraints*.

This kind of problem is termed as a *constraint satisfaction problem* (CSP).

The cats and mice problem is an example of CSP. The constraint on either bank of the river

The number of mice $\geq$ the number of cats, or

The number of mice = 0.

CSPs can be solved by general-purpose search algorithms. However, due to their special structures, algorithms designed specifically for them generally perform much better.

Informed / Heuristic Search

Suppose the problem is to solve the following 8-puzzle problem:

1	2	3
4	5	
7	8	6

Initial State

1	2	3
4	5	
7	8	

Goal State

It can be solved by only one move. However, if a depth first search is unlucky, it could pick LEFT or UP to try next (move the blank to left or up).

This would result in worse positions than the starting position. It is possible that the search will never try DOWN and find the right move.

To efficiently solve a problem, we can force the search to try the best or the most promising move first.

This can be done by using *domain knowledge* or *heuristic information* to guide the search.

This kind of search strategies is called *informed search* or *heuristic search*.

Heuristic search applies a *heuristic function $h(n)$* to a node n . It measures “how promising the node is”, or the desirability of the node. The lower the heuristic value, the more promising or more desirable the node is.

Pseudo Code of Informed Search

```
/* OPEN and CLOSED are lists of states. */  
OPEN = start node, CLOSED = empty
```

```
While OPEN is not empty do
  Remove the first state from OPEN, call it X

  If X is a goal return path from start to X
  Put X on CLOSED
  Generate all successors of X

  For each successor of X do
    DEAL_WITH_IT(successor)
    Re-order the nodes on OPEN by value of f
  End for
End while
```

```
DEAL_WITH_IT (successor)
  case: the state of successor is not on OPEN or CLOSED
    assign the successor a value by function  $f$ (successor)
    add successor to OPEN
  case: successor is already on OPEN
    if its state was reached by a shorter path here
    then give the state on OPEN the shorter path
  case: successor is already on CLOSED
    if its state was reached by a shorter path here
    then
      remove the state from CLOSED
      update path
      add the successor to OPEN
    end case
```

Evaluation Function

The function $f(n)$ in pseudo code of informed search is evaluation function. An evaluation function can be applied to any node. Nodes on OPEN are sorted by return value of the evaluation

function. The node with the smallest evaluation value is expanded next. Different choices for $f(n)$ give different kinds of searches.

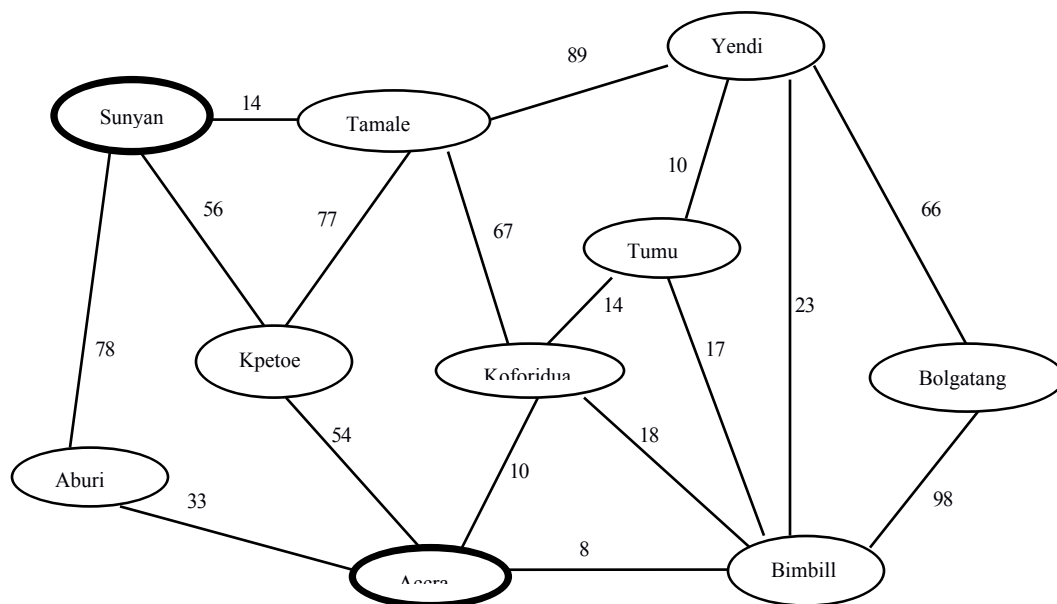
If $f(n) = h(n)$, then it is a greedy search. [The evaluation function is a heuristic function]

If $f(n) = h(n) + g(n)$, then it is an A* search. [It combines heuristic function and path cost function].

Question: If $f(n) = g(n)$, what kind of search is it?

Greedy Search

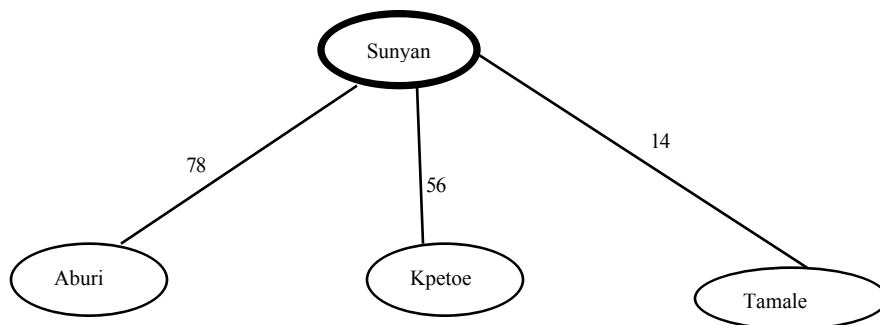
Greedy search uses heuristic function as an evaluation function. It expands the node that appears to be closest to the goal.



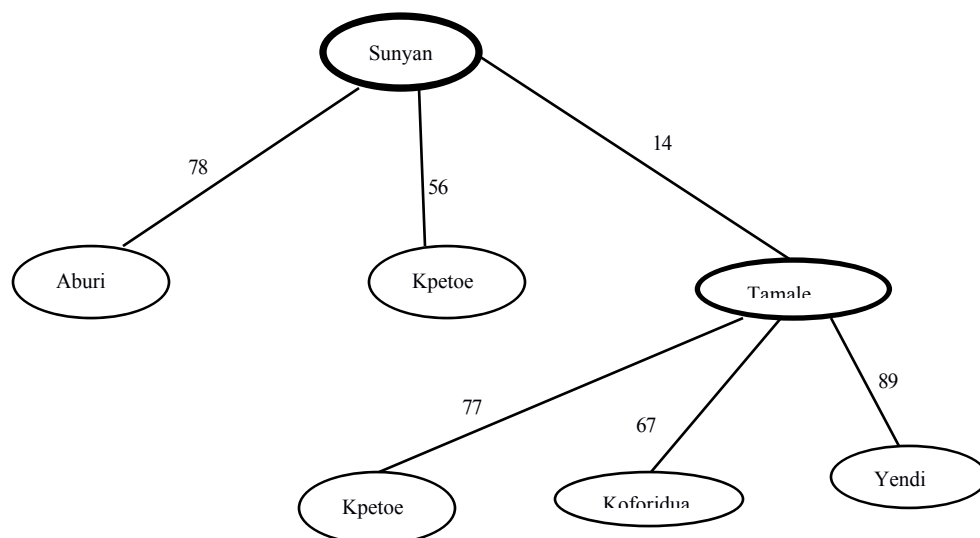
Depth 0:



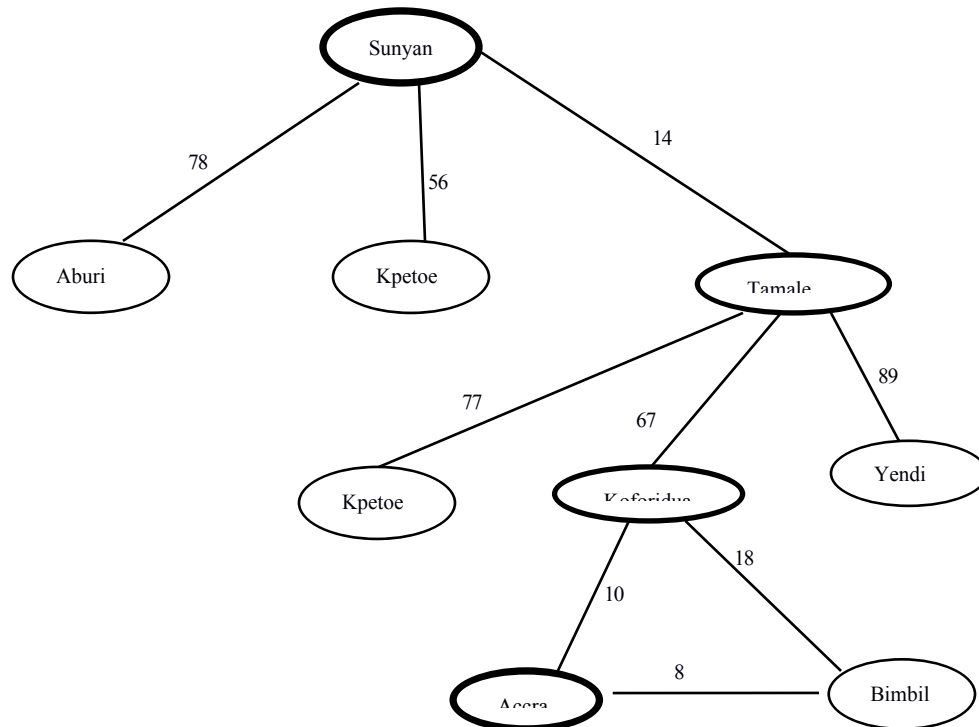
Depth 1:



Depth 2: Tamale is expanded because it has the least heuristic value



Depth 3



Properties of Greedy Search

It is not optimal. Like depth first search, it prefers to follow a single path all the way to the goal. The path might not necessarily be the shortest one.

It is complete only if repeated states can be removed. Otherwise, it can get stuck in loops.

In the sense of time complexity, it is $O(b^m)$ but a good heuristic can give a dramatic improvement.

Regarding space complexity, it is also $O(b^m)$ as it keeps all nodes in memory.

A* Search

A* uses heuristic function and path cost function as evaluation function.

$$f(n) = h(n) + g(n)$$

The idea is to try to expand a node that is on the least cost path to the goal.

$f(n)$ is the estimated cost of the cheapest solution through n .

A* search is complete and optimal. It is said to be *Optimally Efficient* as no other algorithm using $h(n)$ is guaranteed to expand fewer states.